

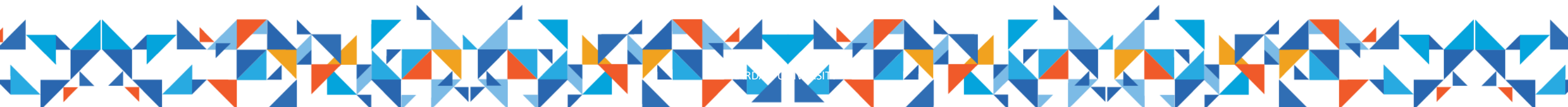


Program counter.

What it is: A register that holds the address of the next instruction to be executed.

Purpose: Allows the CPU to resume the process from the exact place it left off during context switching.

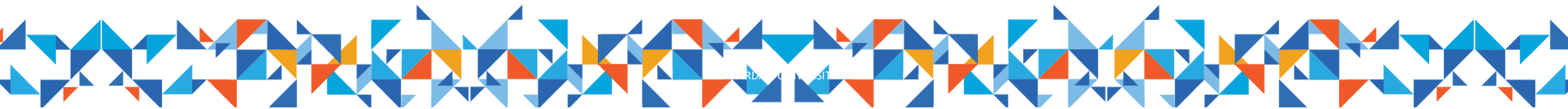
Example: If a process was executing instruction #1500 when interrupted, the PC will store that address in the PCB.





Process Priority:

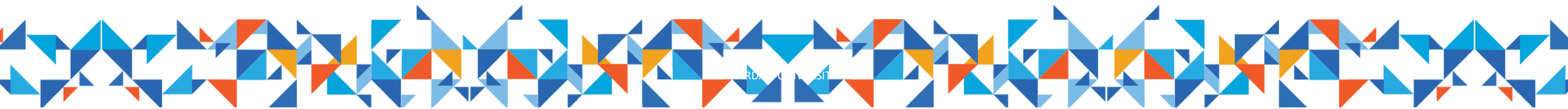
- Process priority is a numeric value that represents the priority of each process.
- The lesser the value, the greater the priority of that process.
- This priority is assigned at the time of the creation of the PCB and may depend on many factors like the age of that process, the resources consumed, and so on.





PCB pointer:

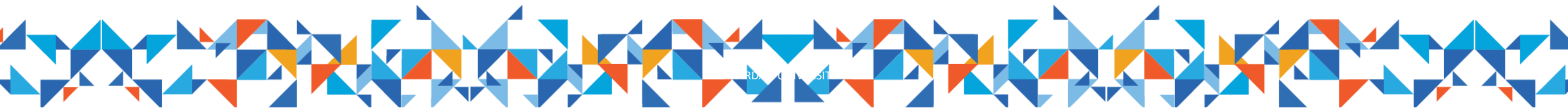
- This field contains the address of the next PCB, which is in ready state.
- This helps the operating system to hierarchically maintain an easy control flow between parent processes and child processes.





List of open files:

- As the name suggests, It contains information on all the files that are used by that process.
- This field is important as it helps the operating system to close all the opened files at the termination state of the process.

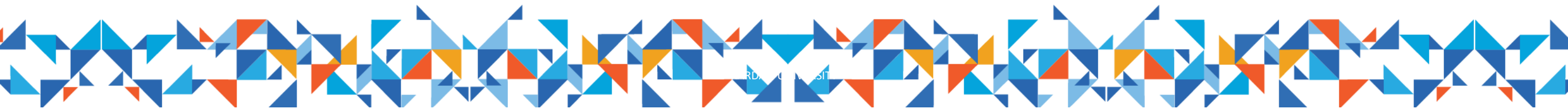




CPU Registers:

What it is: Includes general-purpose registers (like AX, BX), **stack pointer, index registers, status flags, etc.**

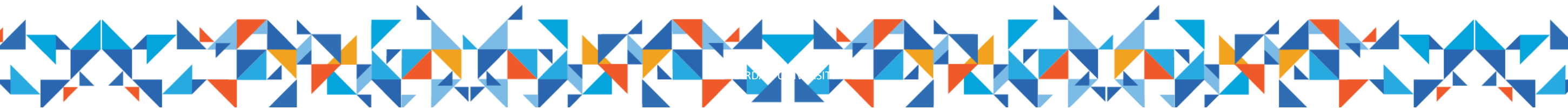
Purpose: Maintains the execution context of the process.
During context switching, all these registers are saved/restored to/from the PCB.





CPU Registers:

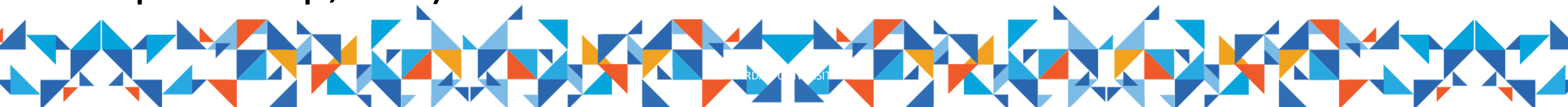
- **General-purpose registers** (AX, BX, etc.) – for math and logic
- **Stack pointer** – tells where temporary memory is
- **Index registers** – for moving through lists
- **Status flags** – for results (like “was the result zero?”)





Example: A Bus Driver and Passengers

- Imagine a bus driver is running a **city shuttle service**, and he switches between multiple routes (Route A, Route B, etc.) throughout the day.
- Each **route** is like a **process**.
- Each **route** has:
- A specific **starting point**
- A **list of passengers**
- A set of **instructions** (where to stop, how many passengers to pick up or drop, etc.)





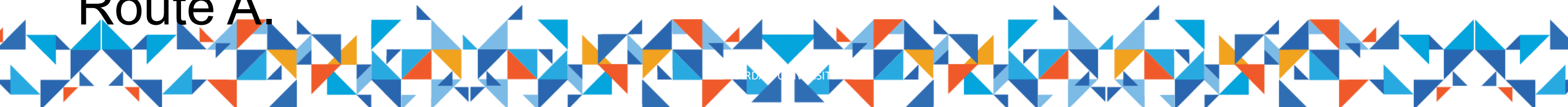
Here's how it works:

- The driver starts with **Route A**.
- He picks up some passengers, makes a few stops — but then he gets a message:

"You have to pause Route A and do Route B now!"

- So, the driver **writes down** all important things:
- Where he stopped on Route A
- How many passengers are on board
- Which stops are done and which are left

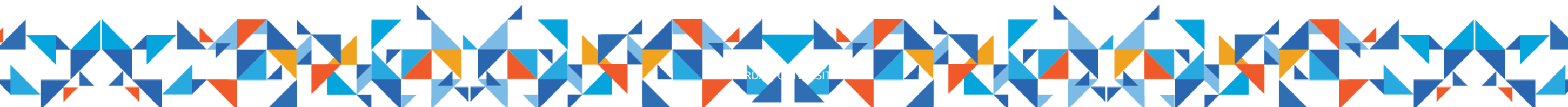
👉 That's like **saving CPU registers into the PCB** of Route A.





Example...

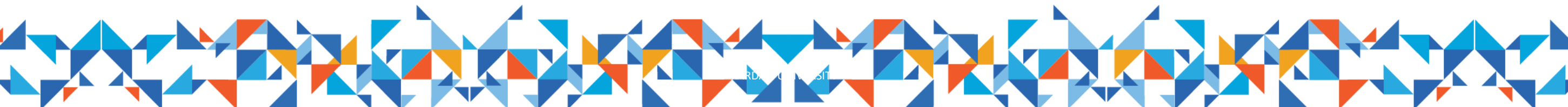
- Now the driver switches to **Route B**, and he uses a different set of instructions and passengers.
 - Later, when it's time to go back to Route A,
 - He pulls out the **notes he saved earlier**,
 - Loads the same passengers,
 - Resumes the exact stop he left at.
- 👉 That's like **restoring the CPU registers from the PCB** and continuing the process.

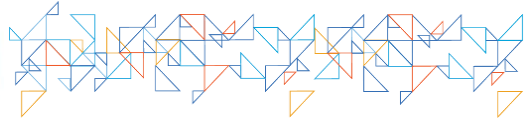




Memory Management Information:

- When a program (process) runs, it needs **some space in your computer's memory (RAM)** to do its job.
- The OS gives it **its own section of memory** so it doesn't mix with other programs.
- But how does the OS remember **which part of memory belongs to which process?**
- That's where **Memory Management Info** in the **Process Control Block (PCB)** comes in!

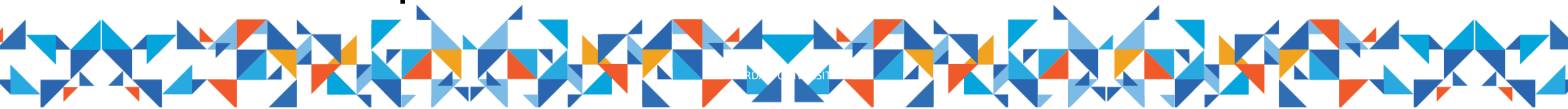




I/O Status Information:

What it stores:

- List of open files
- I/O devices allocated (printers, disk, network)
- I/O request status
- Purpose:** Helps the OS track which files/devices a process is interacting with.
- Example:** A media player's process will have audio hardware registered in its I/O info. If the player crashes, the OS can free up that device.



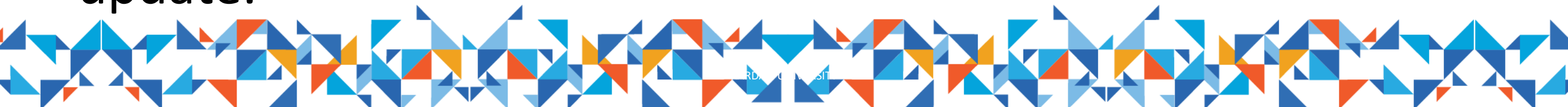


Scheduling Information:

What it includes:

- Process priority
- Scheduling queue pointers
- Scheduling parameters (time slice, etc.)
- **Purpose:** Enables the scheduler to manage the order and time each process gets on the CPU.

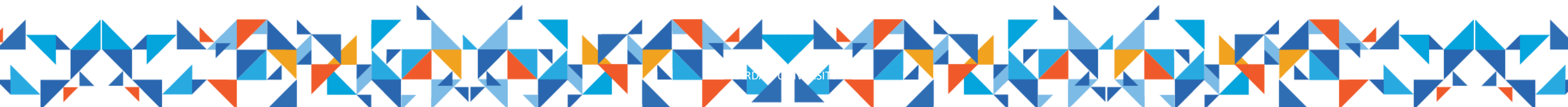
Example: A high-priority process (like antivirus scan) may be scheduled more frequently compared to a background update.





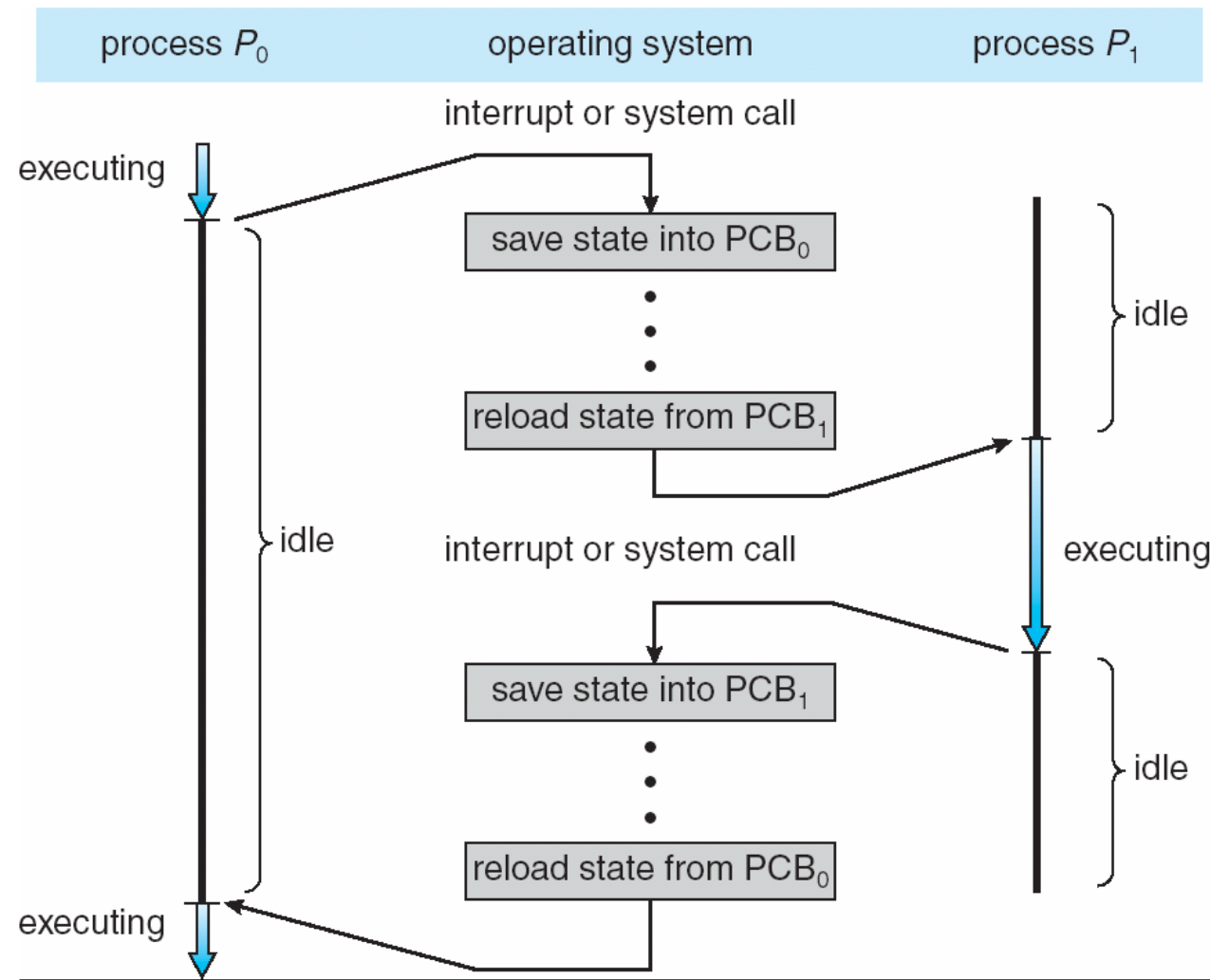
Where is the PCB Stored?

- PCBs are stored in the **Operating System's memory (Kernal) space**: usually in a **process table** maintained by the OS.
- Each active process has **one PCB**.
- The OS can quickly **reference any process's status** or resource usage by accessing the process table.





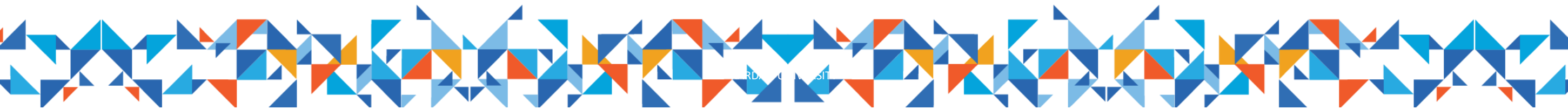
CPU Switch From Process to Process





What Are Process Scheduling Queues?

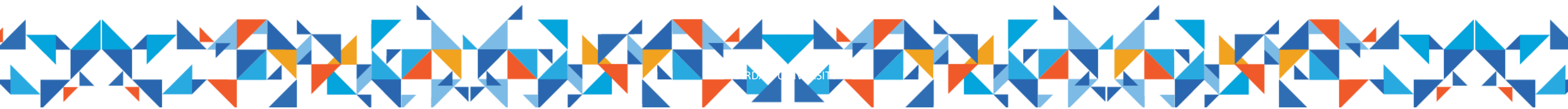
- In an operating system, **processes don't run immediately**.
- Instead, they **wait in lines (queues)** until it's their turn to use the CPU or some other resource.
- These **queues** help the **Operating System manage multiple processes smoothly**.





Why Use Queues?

- Because many processes may be waiting for:
- The **CPU** to execute instructions
- **Input / Output (I/O)** operations
- **Memory** to be free
- Or they may be **blocked** (paused)
- OS puts them into **different types of queues** based on their situation.





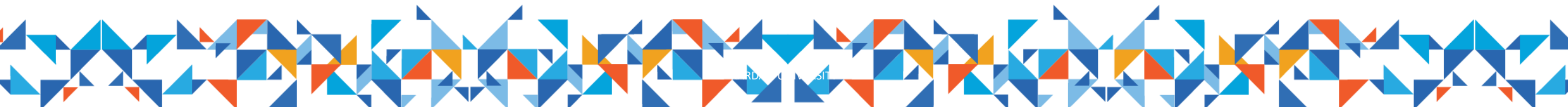
Types of Scheduling Queues

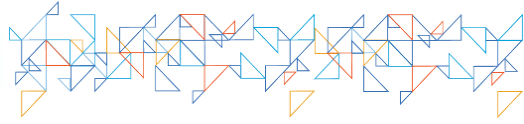
1. Job Queue

All processes that **are waiting to enter main memory (RAM)** for execution.

Simple Example:

Imagine students standing outside the classroom, waiting for their turn to come in and take the exam.





Types of Scheduling Queues

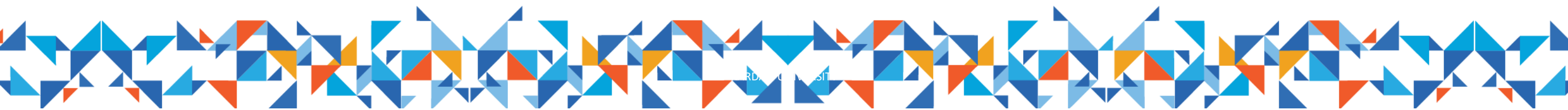
2. Ready Queue

All processes that are **loaded in memory and are ready to run**, but are **waiting for the CPU**.

Managed by the **CPU Scheduler**

Example:

Think of people standing in a line for their turn at the ATM. They are ready and waiting, just not at the machine yet.





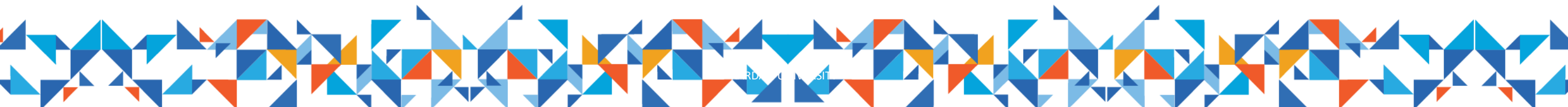
Types of Scheduling Queues

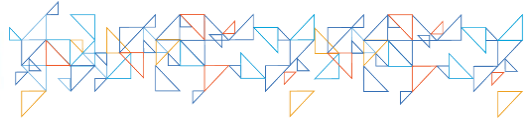
3. Device (I/O) Queue

Processes that are **waiting to use an I/O device** (like printer, disk, etc.)

Example:

Like waiting your turn to use the office printer.





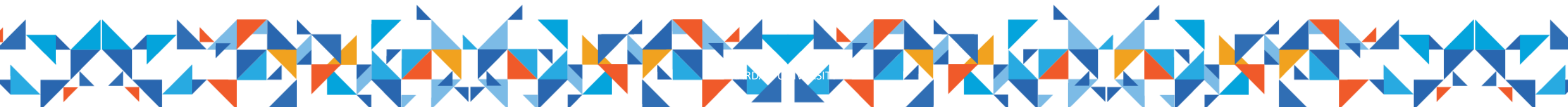
Types of Scheduling Queues

4. Waiting or Blocked Queue

These processes are **not ready** to run because they are **waiting for an event** (like data to be received from disk).

Example:

Like a chef waiting for ingredients before they can continue cooking.





Ready Queue And Various I/O Device Queues

